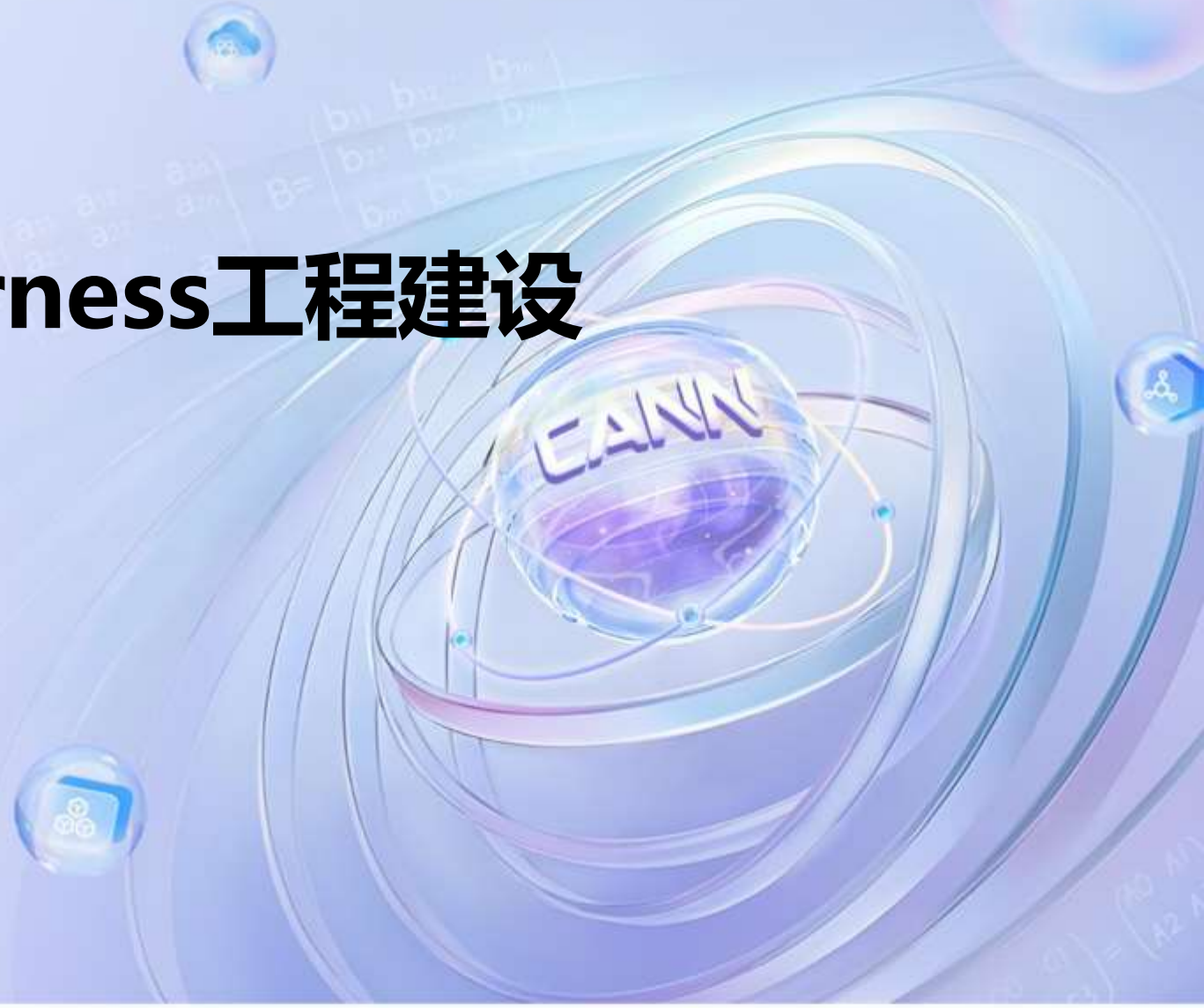
CANNBot 算子Harness工程建设



https://gitcode.com/cann

CANN

修订记录

版本号	修订日期	修订人	修订页码	修订内容摘要
V2.0	2026/08/26	梁冰龙	全部	初版内容

目录

Part 1 Harness 架构设计理念

Part 2 需求分析与设计阶段

Part 3 开发 · 测试 · 自动化门禁

Part 4 全自动化闭环与实践效果

CANNBot 三层架构与 Harness 实现

TEAMS 应用编排层

ops-direct-invoke
Kernel直调开发流程

ops-direct-invoke-flash
Kernel直调 Flash工作流

ops-registry-invoke
算子注册调用开发流程

AGENTS 角色执行层

方案设计 | 代码开发 | 代码检视 | 代码测试

SKILLS 知识能力层

知识库类
NPU 架构与芯片映射
Tiling 设计方法论
API 最佳实践
精度标准

工程模板类
注册转直调模板
直调工程模板

调试与测试类
精度调试速查
运行时/崩溃分析
设备查询与环境验证

测试开发类
ST 用例设计 / UT 开发与覆盖

工具辅助类
性能采集与分析

Teams 层 — 流程编排引擎

阶段递进式工作流，每阶段明确输入 / 输出 / 退出条件
门控确认 CP1 → CP2 → CP3 → CP4 → CP5，逐级审批
机器校验门禁：spec 9-stage + workflow validator 证据对账
自动评审闭环：自审 13 条 → 方案评审 → 测试评审
并行调度：同次响应并发启动多子 Agent，减少串行等待

Agents 层 — 专职角色并行协同

方案设计：需求分析、方案设计、spec 生成与自审
代码开发：代码实现、穿刺验证、UT 开发、汇合联调
代码测试：ST 用例开发、测试设计评审、精度 / 性能验收
代码检视：代码检视、规范检查、设计一致性验证
方案设计与测试设计并行执行，评审 Agent 独立运行互不等待

Skills 层 — 知识外化与注入

领域知识文件化：模式、陷阱、API 指南外化为 references/
工程模板固化最佳实践，统一目录结构、命名与代码规范
调试工具按领域分类，按需挂载到对应 Agent 上下文
自进化闭环：每次故障 → 规则 → 下次自动拦截
新人接入即获老工程师经验，降低学习曲线

https://gitcode.com/cann

CANN

harness工作流

1 阶段一：需求与设计

⛔ CP1 → spec生成(9-stage) → 自审(13条款) → CP2

2 阶段二：开发（三迭代 • 双轨道并行）

A1主线 + A2(UT) + B(ST) + A1-P穿刺验证

3 阶段三：验收

⛔ 精度验收 | 🟣 性能验收

4 阶段四：上库

🟣 CP5确认 → git merge

多 Agent 协同体系

🏗️ 方案设计Agent
需求分析 · 方案设计 · spec生成与评审 · 方案评审

💻 开发Agent
代码开发 · UT开发 · 穿刺验证 · 汇合联调 · 性能优化

🧪 测试Agent
ST开发 · 测试设计 · 精度验收 · 性能验收

🔍 代码检视Agent
规范检查 · 设计一致性验证

核心工作流原则

- ✅ 测试驱动：验收标准先行，功能实现在后
- ✅ 阶段递进：骨架→整合→全量，禁止跳阶段
- ✅ 阶段门控：每阶段必须通过验证方可进入下一阶段
- ✅ 设计/测试锁定：审批后锁定，变更需审批并更新
- ✅ 并行协同：方案设计与测试设计并行执行
- ✅ 自进化闭环：每故障 → 固化Hook/YAML模式

https://gitcode.com/cann

CANN

目录

Part 1 Harness 架构设计理念

Part 2 需求分析与设计阶段

Part 3 开发 · 测试 · 自动化门禁

Part 4 全自动化闭环与实践效果

阶段一（上）：需求分析与 Spec 生成

1.1 开发准备
创建算子目录 /初始化 LOG.md
创建 issues /目录 Git 分支创建

↓

1.2 需求分析
生成需求分析文档 REQUIREMENTS.md

↓

⛔ CP1 确认
用户确认需求分析摘要 / 修订后重新确认

↓

1.2.5 spec 生成
生成 spec.yaml 数学契约 / 9-stage 机器校验 / FAIL 自动重试 ≤2次

↓

1.2.5R spec 自审
13条 SPEC-* 条款级评审
生成 SPEC_REVIEW.md

9-Stage 校验内容
Stage 1-2: YAML 结构 + 必填字段校验
Stage 3-4: dtype 矩阵 + shape 约束校验
Stage 5-6: invariant + boundary case 校验
Stage 7-8: tolerance + 平台信息校验
Stage 9: 算子仓路径校验（可SKIP）

13条 SPEC-* 评审条款
SPEC-1~3: dtype / shape / 芯片平台 对照
SPEC-4~6: 接口签名 / 错误码 / 性能字段
SPEC-7~9: 资源预算 / 精度目标 / 边界场景
SPEC-10~13: 文档一致性 / 引用完整性
机器可判项对照 REQUIREMENTS ↔ spec 自动逐条

失败处理与重试
9-stage FAIL → 自动调用 architect 修订
1.2.5R 状态= ❌ → 回 1.2.5 重跑全流程
每种失败最多重试 2 次
超过 → 创建 issue 文件并汇报用户
spec 未通过禁止进入 1.3 / 1.4

CANN

阶段一（下）：方案设计与测试设计

1.3 方案设计 || 1.4 测试设计（并行）→ 评审 → CP2 确认

1.3 方案设计

输入: REQUIREMENTS.md + spec.yaml
生成详细设计文档 DESIGN.md
含 spec.yaml 一致性映射
字段冲突按所有权规则处理

1.4 测试设计

输入: REQUIREMENTS.md + spec.yaml
生成测试设计文档 TEST.md
含 spec.yaml 测试映射
黑盒用例按 ascendc-st-design 流程

为什么并行执行？

两者都以需求为共同真值源
避免 REQUIREMENTS 双方各自解读导致漂移
节省时间：设计+测试同时完成
dtype/shape/invariant 全部机器化锁定

1.3R 方案评审

scene: design-review
状态=❌ → 回 1.3 修订重跑
冲突 → 流程终止报用户

1.4R 测试设计评审

scene: test-design-review
状态=❌ → 回 1.4 修订重跑
冲突 → 流程终止报用户

评审失败处理

非冲突原因：自动修订 → 重跑评审
最多重试 2 次，超过 → 创建 issue
REQUIREMENTS/spec 冲突 → 流程终止
禁止把 ❌ 报告直接抛给用户,禁止评审未通过触发 CP2

⛔ CP2 用户确认

前置：1.3R=✅ 且 1.4R=✅

阶段二：三迭代开发·双轨道并行

1 迭代一：骨架搭建

目标：单TilingKey骨架
第一波并行：
A1-Main：单TilingKey开发
A1-P：穿刺验证其他分支 B
：L0标准用例开发
第二波（编译通过后）：
A2：核心路径UT
A1-P-Retry：失败穿刺重试
汇合验证 → 测试工程师验收

2 迭代二：策略整合

目标：多TilingKey整合
第一波并行：
A1-Main：整合迭代一结果
A1-P：穿刺验证迭代三任务
B：C++多shape用例
第二波（编译通过后）：
A2：Tiling分支UT覆盖
A1-P-Retry：失败穿刺重试
汇合验证 → 测试工程师验收

3 迭代三：全量覆盖

目标：全功能实现
第一波并行：
A1-Main：整合迭代二结果
B：C++全量用例开发
第二波（编译通过后）：
A2：全覆盖UT
汇合验证 → 验收
* 白盒测试生成 → PyTorch ST

⛔ 禁止逐个启动A1-P | 每迭代第一波必须同次响应并发 | 汇合验证 ST通过率=100% | 测试工程师验收 通过率=100% | 禁止 CPU Mock 冒充 NPU 验证

目录

Part 1 Harness 架构设计理念

Part 2 需求分析与设计阶段

Part 3 开发 · 测试 · 自动化门禁

Part 4 全自动化闭环与实践效果

自动化校验门禁

机器校验 + 自动评审 + 证据对账，三层自动化保障

Level 1: spec 9-stage 校验

python3 validate_spec.py spec.yaml
9 个阶段逐项机器校验
Stage 1-2: 结构 + 必填字段
Stage 3-6: dtype/shape/invariant/boundary
Stage 7-9: tolerance/平台/路径
Stage 9 SKIP 视为通过

Level 2: 自动评审闭环

1.2.5R spec自审: 13条 SPEC-* 条款
1.3R 方案评审: design-review scene
1.4R 测试设计评审: test-design-review scene
每条评审 FAIL → 自动修订重跑
每种失败最多重试 2 次
超过 → 创建 issue 并汇报用户

Level 3: workflow validator 证据对账

python3 validate_workflow_state.py
--stage cp2: 开发期黑盒证据校验
--stage cp3: 最终黑盒/白盒证据校验
以机器证据为准，Markdown摘要不能替代
STATUS: PASSED 方可推进

校验前置，禁止跳过

spec 未通过 → 禁止进入 1.2.5R
spec自审未通过 → 禁止进入 1.3/1.4
方案评审未通过 → 禁止触发 CP2
测试评审未通过 → 禁止触发 CP2
validator 未 PASS → 禁止触发 CP3

重试与兜底策略

所有自动重试上限 2 次
超过 → 创建 issue 文件归档
主 Agent 可直接 Write issue 文件
SPEC-* / DESIGN-* / TEST-* 冲突 → 终止
禁止静默处理，禁止直接抛给用户

证据链完整性

CP2: 开发期黑盒测试证据
CP3: 最终黑盒 + 白盒(high/full)证据
Markdown 数字 = 机器证据数字
不一致 → 校验失败
所有证据可追溯、可审计

https://gitcode.com/cann

CANN

测试体系

C++ ST 快速验证 + PyTorch ST 最终验收 + 白盒测试生成 + 证据校验

C++ ST 测试（迭代验收）

B 任务产出，随迭代一/二/三逐级扩展
迭代一：L0 标准用例（基础 shape + 单 dtype）
迭代二：多 shape 用例（C++ 快速验证）
迭代三：全 dtype + 边界 + 广播用例

PyTorch ST 测试（最终验收）

C 任务独立产出，阶段二末一次性完成
torch_adapter.cpp + golden.py + test.py
L0 + L1 全量用例，一次性开发
编译通过 + CPU Golden 自测
最终精度验收使用（阶段三 3.1）

白盒测试（结构覆盖）

触发时机：迭代三验收通过后，精度验收前使
用 ascendc-whitebox-design skill 工作流产
出 high case set + full case set
通过 workflow validator 对账验证
验收标准包含白盒证据校验

测试驱动开发 (TDD)

测试设计先于代码实现
1.4 测试设计 与 1.3 方案设计 并行
黑盒用例按 spec.yaml 生成，不依赖实现
桩使测试失败 → 实现使通过
所有测试报告放 tests/reports/

证据校验红线

禁止 CPU Mock 冒充 NPU 验证
ST 通过率必须 = 100% 方可推进
Markdown 数字必须与机器证据一致
CP2: validate_workflow_state.py --stage cp2
CP3: validate_workflow_state.py --stage cp3

测试覆盖矩阵

L0 用例：基础 shape + 单 dtype（边界值）
L1 用例：多 shape + 全 dtype + 广播
白盒 high：关键分支 + 异常路径
白盒 full：全分支 + 全边界 + 等价类
三层交叉覆盖，消除测试盲区

CANN

目录

Part 1 Harness 架构设计理念

Part 2 需求分析与设计阶段

Part 3 开发 · 测试 · 自动化门禁

Part 4 全自动化闭环与实践效果

CANNBench 闭卷评测总榜

💡 ops-direct-invoke-flash 模式：端到端生成响应快，开发者即时反馈体验好。 ops-registry-invoke模式侧重生产级完整交付，流程更长，适合正式上库场景。

Ascend 950PR
ops-direct-invoke-flash: exp / sigmoid / gelu / argmax / softmax / topk / transpose / GQA
8 算子 × 100 分 = 总分 800 分

无 Harness

22.45

/ 100 分 **•得分率 22.45%**
平均每算子仅 22.45 分 **•总分 179.6 / 800**

有 Harness (ops-direct-invoke-flash)

52.27

/ 100 分 **•得分率 52.27%**
五层纵深防御 •自动化检查保障 **•总分 418.14 / 800**

📊 **失分维度 vs Harness 兜底对照**
📊 **CANNBench 三维度评分 • Harness 对口兜底**
编译正确性 20% ← 4类阻断 + reg-api-review
精度正确性 30% ← 多维度Agent review+ 测试驱动开发

https://gitcode.com/cann

CANN

展望

基于 ops-registry-invoke 持续优化，下一阶段方向

1 迭代效率提升

当前三迭代（骨架→整合→全量）串行推进。通过更精准的 TilingKey 预测减少穿刺验证轮次，缩短单算子开发周期。

2 确认点智能化

当前 CP1-CP5 依赖人工逐级确认，流程中多达 5 次人工介入，后期减少人工介入，让开发流程更流畅。

3 端到端可观测

当前靠 LOG.md + git tag 手动追踪，缺乏全局视角。目标：自动生成开发报表——各阶段耗时分布、失败热点分析、重试统计。用数据回答：哪个阶段卡最久？哪个校验失败最多？哪里值得优化？数据驱动流程改进，而非凭感觉调整。

Thank you.



群聊：CANNBot Skills开源交流
2群



该二维码7天内(6月24日前)有效，重新进入将更新

欢迎加群交流

社区愿景：打造开放易用、技术领先的AI算力新生态
社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

https://gitcode.com/cann

CANN